

internal-only information as an internal note on the ticket.

1. Tag the next on-call engineer in the emergency's Slack thread.
1. Merge into non-emergency ticket to consolidate and link everything.

When the customer incident is resolved

As soon as the customer incident is resolved, mark the emergency ticket as solved. Consider whether an emergency summary is necessary to add in an internal comment. Any follow up work should be in a separate ticket · do NOT continue work within the emergency ticket.

- Option 1: A related ticket already exists:

1. Add an internal comment linking to the (solved) emergency ticket.
 1. Add an internal comment in the emergency ticket, linking to this ticket as the follow-up ticket.
 1. Check that the priority of the follow-up ticket fits.
 1. Write post-call notes (using macro Support::Customer Calls::Call Completed - Summary) relevant to the customer in a public reply on the follow-up ticket.
- Be sure to let the customer know that follow up work will continue in this ticket.

- Option 2: There is no related ticket:

1. Create a new ticket by following the steps provided in this doc How can I open a new ticket on behalf of a customer?.
1. Let customer know in the ticket description that follow-up work will continue in this ticket.
1. Add an internal comment linking to the (closed) emergency ticket.
1. Add an internal comment in the emergency ticket, linking to this ticket as the follow-up ticket.
1. Optionally, an engineer involved in the emergency can take ownership of the ticket.

Why do follow up work in another ticket?

- We are at risk of missing customer responses that may come in to the ticket after the original assignee's shift ends.
- Emergency tickets have a shorter internal NRT SLO to encourage us to respond very quickly.
- Emergency tickets count differently in our statistics.

Optional: create pairing issue

Another option to consider is creating a pairing issue with the label pairing-customeremergency, including the support engineer(s) who provided assistance during the emergency. If your capacity is limited, you can request any engineer who was involved in the emergency to create the pairing issue on your behalf. This way, we can ensure that the necessary collaboration efforts get tracked.

What to do if you don't know what to do

First, remember that your primary role is incident management. You are not expected to have all the answers personally and immediately.

Your primary job is to coordinate the emergency response. That could mean:

- directing the customer to take specific actions
- finding relevant documentation or doing other research into the problem
- identifying a known bug or regression and providing a workaround
- analyzing log data

It could equally mean:

- identifying other experts on the Support team to help do the above
- reaching out to development teams to find a subject matter expert (SME)
- suggesting that the customer reach out to additional experts on their side (for example, if the problem is slow storage, you might suggest getting someone from their storage team)

Remember to say only things that help the customer and that maintain their confidence in you as the person in charge of getting their problem resolved. When you're not sure what to do, you might also be unsure what to say. Here are some phrases that might help:

- What have you done up until now to try to resolve this?
- Please give me a few minutes to check the documentation on that.
- I'm doing some research to find the answer to that; please give me a few minutes.
- I'm working on finding someone who has specific expertise in this area.
- I don't know the answer just yet, but I'm here for you and I will use all the resources at my disposal to get this resolved.

If you encounter a SaaS emergency at the weekend that you are unable to progress, then consider checking if the CMOC engineer on call is available to offer any help or guidance.

If you are still stuck and are having difficulty finding help, contact the manager on-call.

Triggering a Developer Escalation

On rare occasions, you and the manager on-call may decide it's necessary to initiate the developer escalation process to get the needed developer input. Keep in mind that the developer who takes the escalation might not be familiar with the aspect of GitLab that is the focus of the emergency, and it can take them time to get up to speed.

To trigger a developer escalation, see this process outline.

Supporting 24/7 Coverage for Customers on the Advanced or Signature Success Tier - Phase 1

Customer Support provides 24/7 coverage for customers subscribed to GitLab's Advanced and Signature Success Tiers. These premium tiers, which include access to a Customer Success Architect (CSA), require continuous support and faster response times for Severity 2 issues (labeled as High Priority tickets in Zendesk).

Success Tier	Coverage
Advanced	- 24/7 High Priority coverage
Signature	- 24/7 High Priority coverage - 2hr SLA

Weekend Coverage

Commencing May, 2025 @ceoc, managed by our existing on-call rotation schedule, will temporarily be responsible for weekend coverage for these tickets. This means we need to provide a first reply and ensure the customer doesn't get into a critical scenario during the weekend. A team of dedicated engineers hired for the permanent 24/7 roles will use this period for onboarding and training. This is Phase 1 of a 3-Phase approach detailed further in STM6743 24/7 Shift role and its implementation.

PagerDuty Alerts

From 2025-05-01 PagerDuty Alerts for High Priority tickets received from customers with either of the two success tiers will be triggered over the weekend. The CEOC will need to action this ticket within the SLA stated above, dependent on the customer's entitlements, which will be clearly stated in Zendesk.

Ticket Assignment and Handling

We as a company want to treat High Priority tickets, especially from customers with a Success Tier, with urgency and action.

1. When you start your shift take over any emergencies or High priority tickets from the leaving CEOC that needs attention.

1. When you get paged on a High priority ticket work with the customer on Solving the pieces that are causing the ticket to be a high priority, by working towards reducing the ticket's priority.

1. There is no need to Unassign tickets over the weekend. Ticket that needs ongoing attention should be handed over to the incoming CEOC by using the PD alert and tickets in safe state should stay assigned to the CEOC leaving shift.

Customers with an Assigned Support Engineer (ASE)

In cases where:

- An assigned support engineer has opted to automatically assign their customer's tickets to themselves; and
- The customer creates a Sev 2 ticket on a weekend.

This will:

1. Generate a CEOC page for the ticket; and
1. Assign the ASE to the ticket.

CEOC will need to respond to such tickets, even though it may have been assigned to the ASE. You can view all Unsolved, High Priority tickets for Success Tier customers in the Report: All Success Tier Customers' Unsolved Severity 2/High Tickets on the 24-7 Success Tier Coverage Zendesk Explore dashboard.

Once the customer ticket has been de-escalated and is no longer a Severity 2/High Priority issue, reset the ticket priority, and let the customer know we will continue working with them during standard operating hours.

The Support Manager On-Call is always available to support you. Follow these steps to engage the Support Manager On-call for assistance.

Rollout and Monitoring

Estimated timeline for Phase 1: 1-2 months, depending on ticket volume and customer adoption.

Zendesk Explore Dashboard: 24-7 Success Tier Coverage

This is being closely monitored by Manager DRIs (@erikamiklos (EMEA), @ralfaro (AMER), @kslaats (APAC)) in the Zendesk Dashboard: 24-7 Success Tier Coverage.

License Emergencies

During the week

For license emergencies during the week, reach out to supportlicensing-subscription and ask for an expert there to handle the case. Ping the current Support Manager On-call in your request so they can ensure it gets picked up. Once pinged, the Support Manager On-call is the DRI for ensuring the emergency gets handled.

In the event the customer's instance is unusable due to an expired license and you are unable to reach the L&R team or the on-call manager, generate a trial license.

On a weekend

Self-managed Subscription Emergencies

There may be times when a customer's subscription expires over the weekend, leaving their instance unusable until a new subscription is generated.

For non-trial subscriptions, you can remind the customer that subscriptions have a 14-day grace period. If the grace period will still be active on the next business day, kindly let the user know that their request will be handled as a standard L&R case during normal business hours. You should close the emergency ticket and ask the customer to open a new L&R ticket in case one doesn't exist yet.

Otherwise, follow the Self-Managed Weekend Emergencies - License Request workflow.

SaaS Subscription Emergencies

Subscription expired and downgraded to Free

A customer may be blocked because of a license expiring or neglecting to apply a renewal. If this happens over the weekend:

1. Look up the namespace details using chatops or a GitLab.com Admin account by using the namespaces API (<https://gitlab.com/api/v4/namespaces/<NAMESPACE>>)
1. Check the Trial ends on date.
 - If it has a date, you will not need to provide a Subscription Name in the next step.

Proceed to step 3.

- If it is empty or null and the namespace is on a Free plan, guide the customer to navigate to the Settings -> Billing page and click on Start a Free Ultimate trial.

1. In the ticket, apply the Trial Subscription - Exclusions Sign Off macro and send the generated message to the customer. Do not proceed further until the customer has provided a written response confirming they understand the trial subscription exclusions.

1. Access the CustomersDot Support Admin Tools use the Trial changes (SaaS) option to resolve the situation.

- Search for the namespace ID.
- Select the pencil icon to edit the trial.
- Select the Plan the customer had initially purchased, or use Ultimate if you do not have this information.
- Set the end date to 10 days later.
- Add the relevant Zendesk ticket link to Zendesk ticket link field
- Click Save.

1. When the customer confirms, close the emergency ticket.

1. Alert supportlicensing-subscription by linking to the ticket for follow-up.

Multi-year subscriptions

- Note: Refer to Handling multi-year subscriptions for Self Managed licensing.

Some legacy-type subscriptions are called "multi-year" but are actually multiple, separate subscriptions sold up front to cover a multi-year period. When a customer has a deal like this, each anniversary of the renewal requires that the next subscription be associated, or else the customer's namespace can fall back to Free. If this occurs, send the customer the following steps.

1. Ask the customer to login to the Customers Portal located at <https://customers.gitlab.com/customers/signin> for subscription management.

1. Follow these steps to ensure their GitLab.com account is linked.

1. Follow these steps to update the linked namespace.

If the customer's CustomersDot account is not linked to their GitLab.com account (uid and Gitlab user in the CustomersDot account is empty) and you need to act on their behalf, you can try Force Association of the subscription as a workaround.

If neither of the above resolves the issue for the customer, contact the on-call Support Manager requesting further guidance.

SaaS Emergencies

The workflow for these calls is the same as with self-managed emergencies: success means that the customer is unblocked. In some cases, you may even be able to fully resolve a customer's problem.

For any customer facing a SaaS Emergency you are empowered to perform any two-way door action required to unblock them without seeking approval first.

Some examples:

- manually setting a subscription level
- adding additional storage
- adding extra compute minutes
- toggling a feature flag

During a SaaS Emergency, you have additional visibility into problems that a customer may be facing.

Review:

- Using Kibana - explore GitLab.com log files to find the errors customers are encountering.
- Using Sentry - get access to the full stacktrace of errors a customer might encounter.

We're expecting, broadly that emergencies will fall into one of five categories:

- broken functionality due to a regression being pushed to GitLab.com
 - Success may mean: reproducing, identifying or creating a bug report and escalating to have a patch created and deployed.
- broken functionality due to an inconsistency in data unique to the customer, for example: a group name used to be able to have special characters in it, and now something broke because our group name has a special character in it.
 - Success may mean reproducing the error, identifying it Sentry/Kibana, escalating to have the specific data corrected (and creating a bug report so our code is better)
- GitLab.com access or "performance" degradation to the level of unusability, for example: no access in a geographical area, CI jobs aren't being dispatched. This is the hardest class, but will generally be operational emergencies.
 - Success here means making sure it's not actually one of the top two before declaring an incident and letting the SRE team diagnose and correct the root cause.
- License / Consumption issues are preventing access to the product
 - Success here means getting the customer into a state where they're unblocked and making sure the license team is equipped to take the handover.
- a widespread incident causes multiple, successive PagerDuty alerts
 - Success here means tagging and bulk responding to the issues pointing to the GitLab.com Status Page and production issue.

Broken Functionality

If a customer is reporting that behaviour has recently changed, first check GitLab.com Status and incidents for any on-going incidents. If there's no known incident:

1. Initiate a call with the customer. You're specifically looking to:
 - observe broken behavior.
 - determine if there's a known issue, bug report, or other customers reporting similar behavior.
 - ascertain whether or not a feature flag that may have been recently turned on (see: Enabling Feature Flags on GitLab.com)
 - find/build reproduction steps devoid of customer data to build a bug report if none exists.

Broken functionality due to a regression or feature flag

1. Create a ~"type::bug" issue and have the customer review it.
1. Escalate the ~"type::bug" issue
 - If it's a new bug, or a bug with S1/S2 severity escalate using the InfraDev Escalation Process. In most cases we will generate a roll-back patch and apply it to GitLab.com.
 - If it's a feature flag, work with the who turned it on to disable it through ChatOps. In some cases, you may need to use the InfraDev Escalation Process to raise a developer.
1. If this is affecting multiple customers, declare an incident to engage the incident response team who will update the status page.
1. Once the original functionality is restored, update the customer.

Broken functionality due to something specific to the customer

1. Page the Support Manager on-call to review the best way to unblock the customer. It may be that you will need someone with .com console access to fully investigate / resolve.

Broken functionality due to an incident

If there is a known incident, it's acceptable to link to the public status page and related incident issue. Consider using Support::SaaS::Gitlab.com::Incident First Response.

Example tickets

- Feature flag broke previously working behaviour: resolution was to turn off a feature-flag.
- Regression on GitLab.com broke previously working pipeline: resolution was to revert a recently deployed MR.
- Customer locked themselves out of their group by changing SAML settings

Consumption Issues

Quota of compute minutes is blocking a production deployment

A customer may be blocked because they have run out of compute minutes.

1. Advise them to purchase additional compute minutes or set up individual runners.
1. At your discretion, as a courtesy, set an additional 1000 compute minutes on their namespace through ChatOps

Customer has exceeded their storage quota

A customer may be blocked because they've exceeded their storage quota.

1. Advise them to purchase additional storage
1. In cases where a customer is unable to complete a purchase because of a defect or outage, as a courtesy, someone with GitLab.com admin can override the storage limit on a group.

A widespread incident causes multiple, successive PagerDuty alerts

If an incident occurs on GitLab.com and hasn't been posted on the status page, SaaS customers

may raise emergencies in bulk.
Success in such a situation is two-fold:

1. Route customers reporting the incident to our status page, @gitlabstatus on Twitter and the production incident issue.
1. Sort through the alerts to ensure that there are no emergencies raised that are unrelated to the on-going incident.

If this occurs:

1. Don't panic! Slack and PD alerts may come quickly and frequently. Consider silencing both temporarily and focus on ZD.
1. Verify that an incident has been declared and that the incident is actively being worked.
1. If there is no update on the status page yet, advocate for urgency with the CMOC.
1. Choose a unique tag that will help you identify tickets, using the incident number would be typical. For example: incident-12345
1. Create a bulk response that points to the incident on the status page, @gitlabstatus on Twitter and the production issue. If any of these aren't available yet, you can send a response without to keep customers informed. You can include them in a future update.
 - Share the response that you draft or otherwise coordinate with supportgitlab-com and others fielding first responses. There are likely non-emergency tickets being raised about the incident. Using the same response increases the efficiency with which we can all respond to customer inquiries about the problem.
1. Create the tag by typing it into the tag field of at least one ticket and submitting it - if you don't, it won't show as available in the bulk edit view of Zendesk.
1. Use Zendesk search to identify customer-raised emergencies:
 - priority:urgent orderby:createdat sort:desc will show all emergency tickets, sorted by those opened most recently
 - priority:urgent orderby:createdat sort:desc status:new will show all new emergencies
 - CAREFUL: Verify that each ticket is related to the incident - if it is not, follow handling multiple simultaneous emergencies
1. Use Zendesk Bulk Update to respond to all open tickets.

At any point, you may ack/resolve PD alerts. It may be faster to do so through the PagerDuty web interface.

During an incident:

- If there is no production issue to link to yet: let customers know we are actively working to address the problem and that we will follow-up with a link to a tracking issue as soon as one is created. Set the ticket to Open. Once the issue is available, send a follow-up note letting the customer know that they should follow along with the issue and that we are marking the ticket as Solved. Include a note that they should reply if they still have trouble once the production issue has been closed / the incident has been declared resolved.
- If there is a production issue to link to: let customers know we are actively working to address the problem, that they should follow along at the issue, that we are marking the ticket as Solved and they should reply if they still have trouble once the production issue has been closed / the incident has been declared resolved.

Using Zendesk Bulk Update

Zendesk Bulk Update is a way to mass edit and respond to tickets. During an incident, you can use it to:

- automatically tag tickets
- send a bulk response
- set status en masse

You can bulk edit tickets by:

1. From a Zendesk search click one or more checkboxes

1. Click "Edit n tickets" in the upper right-hand corner

1. Edit the properties of the ticket you'd like to update. During an incident that will probably be:

- A public reply
- A ticket tag

1. Click Submit with the appropriate status change

US Government On-call

US Government on-call support is provided 7 days a week between the hours of 0500 and 1700 Pacific Time for severity one issues that arise with premium and ultimate customers who have purchased 12x5 US Gov support. Customers who have opted for the 24x7 ultimate for high and emergency coverage can page at any time.

The current on-call schedule can be viewed in PagerDuty([Internal Link](#)). The schedule is currently split into three, 8 hour shifts which roughly correlate with the dayshift, evening, and overnight team member hours:

- Dayshift: 05:00 - 13:00 PT
- Evenings: 13:00 - 21:00 PT
- Overnight: 21:00 - 05:00 PT

Customers are permitted to submit emergencies via email or via the emergency form in the US Government support portal.

On-call Shift Coverage in US Government

In the event that a Support Engineer needs coverage for a scheduled On-call shift, open an issue in Support Team Meta using the [us-gov-oncall-coverage](#) template.

Dayshift engineers needing coverage on a non-holiday weekday may give the shift to the Support Bot. To do so, open an issue in Support Team Meta using the [us-gov-oncall-coverage](#) template and mention your manager for review. After ensuring that the shift(s) in question do not fall on a weekend or holiday remove the override for your shift in PagerDuty and ensure it falls back to the bot user.

Emergencies outside on-call hours

If a non-24x7 eligible customer submits an emergency case outside the working hours of Government Support the following will occur:

- A slack notification will trigger in the sptus-government channel alerting the team to an off hours emergency and indicating follow-up is needed at the start of business hours
- The Off hours emergency request trigger will inform the ticket submitter that it is after hours and give them the option to either create an emergency case in Global support or wait for US Government support to follow-up at the next start of business hours.

Responding to after hours emergencies

Team members who are working after the 12x5 hours may opt to provide support for customers who are having a production incident at the engineer's own discretion. When addressing these it is important to ensure the following is clear with the customer:

- They are not entitled to 24x7 support based on their subscription
- Emergency support is being provided as a one off exception based on the engineer's availability and future after hours support is not guaranteed

The responding engineer should also add their manager as a follower and indicate in an internal note that after hours support is being provided. This will help ensure the appropriate follow-up occurs with the customer's account team.

US Gov Emergencies in Global

US Government customers with 12x5 support packages are permitted to use the global support portal for after hours emergencies if their organization's policies permit interacting and sharing information with non-US citizens. The US Gov support team cannot confirm or deny whether a specific case belongs to a user or organization entitled to US Government support. There is no requirement or restriction on who can reply to emergencies filed in the global support portal. If a user asks for a US citizen please remind them they are using the global support portal where US citizenship is not guaranteed and let them know if that is a requirement they should use the US Government Support portal for future communication instead.

GitLab Dedicated Emergencies

Emergencies from GitLab Dedicated come through the Customer Emergency On Call rotation. The GitLab Dedicated Handbook has information about working with logs and viewing observability dashboards.

Consider using the @sptfocus-dedicated Slack handle to ping members of the GitLab Support team who focus on GitLab Dedicated for additional assistance.

As appropriate, you can use the section on escalating emergency issues to engage the Engineer on Call for GitLab Dedicated.

Special handling notes

There are a few cases that require special handling. If an emergency page falls in one of these categories please follow these special handling instructions. If you think an emergency is special and not called out below, connect with the Support Manager On-call for help as how best to approach it.

Compromised instances

In the event that an emergency is raised about a compromised instance a call can quickly move well beyond the scope of support.

Use the Zendesk macro Incident::Compromised Instance which expands on the approach below.

The customer should:

1. Shut the instance down immediately.
1. Create a new instance at the exact same version and restore their most recent backup into it.
 - Avoid exposing the new instance to the Internet
 - If they do not have a copy of their gitlab-secrets.json, or if the only backups available are stored in /var/opt/gitlab/backups, mount the volume of the compromised instance to retrieve it.
1. Rotate secrets on the new instance:
 - All secrets contained within the gitlab.rb (such as LDAP/email passwords)
 - All secrets in CI jobs such as API keys or remote server credentials
 - GitLab Runner registration tokens and Runner environment variables
1. If the exploit used to compromise this instance is known, then upgrade the new instance to a version that contains a fix for it or apply any known patches/workarounds.
 - In the case that public access is required by the organization, remove network access restrictions once the new instance is appropriately secured.
1. Retain the compromised instance for forensics and additional data recovery.

Do not offer or join a call without engaging the Support Manager on-call to align and set expectations with the customer through the ticket.

Single user, same day purchases

There have been a few documented cases of folks purchasing a single user GitLab license specifically to raise an emergency. If you encounter such a case, engage the Support manager on-call before offering a call.

Customer Emergency On-Call Training Resources

Customer Emergency Shadow PagerDuty Schedule

The Customer Emergency Shadow Schedule can be used by anyone who wishes to shadow customer emergencies to learn before being Customer Emergency On-Call. To add yourself to the shadow rotation, speak to your manager. To modify your rotation schedule, speak to your manager. To shadow for a short span of days, you can click Schedule an Override, then click Custom duration and then select the time zone and the start and end dates and times before clicking the Create Override button to save the changes. To remove overrides, click the x on the override to be

removed in the list of Upcoming Overrides on the right side of the screen.

SECTION: support/workflows/data_handling.md

Overview

Working directly with customers has the average Support Engineer working with some of the most sensitive data that our business handles. Proper data handling is crucial not only for maintaining customer trust but also for meeting our legal and regulatory obligations. This workflow outlines best practices for Support team members to ensure secure and efficient data handling.

Data Classification Standard

The Data Classification Standard is the most important framework for day-to-day operations in Support. Knowing which data is allowed in which system is vital as you collaborate with colleagues.

Both Data and Systems have data classifications. When data has a classification standard, it represents the impact of disclosing that data. When a system has a data classification standard associated with it, it represents what classification of data is allowed in that system.

Systems within the Tech Stack are authorized to process the level of data associated to them per the Data Classification Standard. Storing or processing sensitive data in an unauthorized system poses a risk to our customer's security and privacy and could have legal and/or regulatory consequence. Examples may include a breach of contract or failure to meet our GDPR or SOX obligations.

References:

- Data classification examples (GitLab Internal Handbook - Team Members only)
- Tech Stack
- GitLab Data Subprocesors

What does it mean for a Support team member?

The standards themselves are important to be familiar with, but for Support generally:

- Be aware of the data classification standard for any data you're working with
- NEVER move data to a system with a lower classification
- Think through confidentiality levels and the SAFE framework when working in public. For example, GitLab.com may be cleared for RED data, but you might still expose that data by posting it in a public project, or forgetting that an issue is public

Examples

The following are examples only - the single source of truth for system classifications is the Tech Stack

Slack (ORANGE)

During the course of working emergency tickets, you may be tempted to copy verbatim troubleshooting data such as relevant logs (potentially RED) or screenshots (potentially RED) from a ticket from Zendesk (RED) into a troubleshooting Slack (ORANGE) thread. This would be inappropriate because RED data is not allowed in ORANGE systems.

Instead, consider sharing the troubleshooting data by:

- Putting the relevant log snippets in an internal note in the ticket
- Sharing the information through a GitLab issue (for example, a Fieldnote issue)

If you see RED data being shared in Slack, ask the team member who posted it to remove it.

GitLab.com (RED)

During the course of discussion issues, you might have a relevant example screenshot that lists a customer name or a quote from a customer directly that identifies them. These are examples of ORANGE data, and GitLab.com is cleared for RED.

However, posting ORANGE data in a public issue would go against the confidentiality levels for that data, so would be inappropriate.

Instead, consider sharing the data by:

- making the issue confidential
- creating a confidential comment

If you see internal-only data being shared publicly, you can:

- edit the comment to remove the data
- toggle the Issue to confidential
- remind the team member who posted it about data classification and confidentiality levels

Building an integration between systems

As a productivity boost, you're considering creating a Slack workflow that will grab any negative customer comments and post them into Slack.

To do this, you're going to scrape Zendesk (RED) using GitLab CI (RED) then post the resulting data into Zapier (YELLOW) which will post in a channel in Slack (ORANGE).

Here, you're moving data between systems that aren't cleared for the classification of data you're likely to be sending.

If you see an integration like this, declare a SIRT.

In addition, if you need assistance with integrations, please reach out to the following teams. For any integrations related to Google, Slack, Zoom, Okta etc., please reach out to the Corp Sec team here. For anything Enterprise application related, please reach out to this team here.

Take-aways

- Know the classification level of data you're working with and which systems are cleared for which data.
- Avoid moving data between systems whenever possible: if you can share data within the system it's already contained in keep it there!
- Don't expose your teammates to data they aren't authorized to see: keep limited access data to those to whom it should be limited.

SECTION: support/workflows/data_processing_addendums.md

Overview

Data Processing Addendums are available to paid subscribers of GitLab.com through their Sales representatives. This workflow details what to do with fully executed (signed) DPAs received from customers.

Workflow

>Note: This workflow only applies if the user has a paid subscription for GitLab.com.

1. Locate the customer's Salesforce account. The correct string begins with 001 and can be found: under the Zendesk organization¹, linked in the automatic note posted to a Zendesk ticket², or in the Edit interface of the CustomersDot³ ([screenshots provided below]). The URL for the Salesforce account record will look like:
<https://gitlab.my.salesforce.com/0014M00001hHV5CQAW>.

1. Under the section "Google Docs, Notes, & Attachments", click "Attach File" and upload the file to the record.

1. Should you be unable to attach the file yourself, please forward it to legal@gitlab.com with a link to the Salesforce account.

Screenshots

- 1.
- 1.
- 1.

Example Tickets

- <https://gitlab.zendesk.com/agent/tickets/142052>

SECTION: support/workflows/debugging_ldap.md

Notes

This assumes an omnibus installation.

See LDAP troubleshooting in docs - [View Docs](#)

Testing the LDAP server

1. Install ldapsearch

bash

Ubuntu

apt-get install ldap-utils

CentOS

yum install openldap-clients

1. Check LDAP settings

Edit the following values to match the LDAP configuration in gitlab.rb

Example LDAP configuration

bash

cat /etc/gitlab/gitlab.rb | grep -A 24 ldap_servers

gitlab_rails['ldap_servers'] = YAML.load <<-'EOS' remember to close this block with 'EOS' below

main: 'main' is the GitLab 'provider ID' of this LDAP server

label: 'LDAP'

host: '127.0.0.1'

port: 389

uid: 'uid'

method: 'plain' 'tls' or 'ssl' or 'plain'

binddn: 'cn=admin,dc=ldap-testing,dc=mrchris,dc=me'

password: 'Password1'

activedirectory: true

allow_username_or_email_login: false

block_auto_created_users: false

base: 'dc=ldap-testing,dc=mrchris,dc=me'

userfilter: ''

attributes:

username: ['uid', 'userid', 'sAMAccountName']

email: ['mail', 'email', 'userPrincipalName']

name: 'cn'

firstname: 'givenName'

lastname: 'sn'

groupbase: 'ou=groups,dc=ldap-testing,dc=mrchris,dc=me'

admin_group: 'gitlabadmin'

EOS

LDAP search switches

- D = Bind DN
 - GitLab config value: binddn: 'cn=admin,dc=ldap-testing,dc=mrchris,dc=me'
- b = Search base
 - GitLab config value: base: 'dc=ldap-testing,dc=mrchris,dc=me'
- w = Password
 - GitLab config value: password: 'Password1'
- w = Port & -h = Host
 - GitLab config value: port: 389
 - GitLab config value: host: 127.0.0.1
- s = Search scope
 - GitLab config value: None
 - Default is sub
 - Using sub "(objectclass=)" will return "all" objects

Get all LDAP objects for baseDN

bash

```
ldapsearch -D "cn=admin,dc=ldap-testing,dc=mrchris,dc=me" \  
-w Password -p 389 -h 127.0.0.1 \  
-b "dc=ldap-testing,dc=mrchris,dc=me" -s sub "(objectclass=)"
```

LDAP Error messages (production.log)

Could not find member DN's for LDAP group

text

Could not find member DN's for LDAP group <Net::LDAP::Entry:0x00000007220388

This usually indicates an issue with the uid configuration value in gitlab.rb

When running ldapsearch you can see what attribute is used for the LDAP username. In the below case the username attribute is uid. Ensure uid: 'uid' in the configuration. The default Microsoft Active Directory username value is sAMAccountName

text

```
dn: cn=user test,ou=people,dc=ldap-testing,dc=mrchris,dc=me  
sn: test  
givenName: user  
uid: test  
cn: user test
```


Cannot find LDAP group with CN 'GROUPNAME'. Skipping

This indicates the admin group name was not found admin group: 'gitlabadmin'. Ensure the group exists in AD and is under the groupbase

LDAP search error: Invalid DN Syntax

This indicates a syntax error with one of the configured DNs. Check the following values, ensure they're the full DN.

- groupbase
- binddn
- base

Testing LDAP - valid for 8.10 >

1. Launch the rails console

```
ruby
gitlab-rails c
```

1. Update the logger level

```
ruby
Rails.logger.level = 0
```

1. Perform a group sync

```
ruby
LdapGroupSyncWorker.new.perform
```

1. Perform a user sync

```
ruby
LdapSyncWorker.new.perform
```

1. All commands:

```
ruby
gitlab-rails c
Rails.logger.level = 0
LdapGroupSyncWorker.new.perform
LdapSyncWorker.new.perform
```

1. Check the console for sync output

Removing exclusive lease - Testing (valid for 8.6 to 8.9)

This is used to force an instant sync of LDAP for testing purposes.

1. Edit any LDAP settings required

1. Edit vi /opt/gitlab/embedded/service/gitlab-rails/lib/gitlab/ldap/groupsync.rb

1. Comment out the exclusive lease section (lines may differ in releases) - [View code](#)

1. Run a reconfigure sudo gitlab-ctl reconfigure This will restart GitLab

1. Launch GitLab Rails console gitlab-rails console

1. Execute Gitlab::LDAP::GroupSync.execute

1. LDAP sync will now run

1. Revert changes to the groupsync.rb file when finished

/opt/gitlab/embedded/service/gitlab-rails/lib/gitlab/ldap/groupsync.rb

Additional testing

1. Start the rails console

```
sh
sudo gitlab-rails console
```

1. Create a new adapter instance

```
ruby
adapter = ::Gitlab::Auth::LDAP::Adapter.new('ldapmain')
```

1. Find a group by common name. Replace UsersLDAPGroup with the common name to search.

1. GitLab 8.11 >

```
ruby
group = EE::Gitlab::Auth::Ldap::Group.findbycn('UsersLDAPGroup', adapter)
```

1. GitLab < 8.10

```
ruby
group = Gitlab::LDAP::Group.findbycn('UsersLDAPGroup', adapter)
```

1. Check memberdns

```
ruby
group.memberdns
```

SECTION: support/workflows/dedicated.md

Overview

GitLab Dedicated, from a support perspective, works as a combination of SaaS and Self-Managed. Customers have full Admin access to the instance, but no access to the infrastructure, nor to the backend configurations. This workflow captures the differences, and details of providing support for GitLab Dedicated.

When working on GitLab Dedicated tickets, a good mental model to follow is to determine if the issue is an Application issue or an Infrastructure issue.

- If you're dealing with an Application issue, that is, the issue is within the GitLab application, then you can treat it like a Self-Managed instance while being mindful of the features that are not available for GitLab Dedicated.
- If it's an infrastructure issue, you'll want to consider engaging with the SREs by opening a Request for Help, checking for incidents, or raising one yourself. The runbooks used by the SREs may be useful additional context.

Use the SaaS, Self-Managed and Dedicated Troubleshooting tables to learn more about the differences between gitlab.com, self-managed and GitLab Dedicated.

If you'd like to work on GitLab Dedicated tickets, consider creating an issue using the template in Support Training, and read the overview.

Below is a list of other GitLab Dedicated Support workflow pages. This list is a temporary measure to workaround the lack of until workflow categories are reintroduced

- GitLab Dedicated Logs
- GitLab Dedicated Observability and Monitoring (Grafana)
- GitLab Dedicated Switchboard Troubleshooting
- Hosted runners for GitLab Dedicated

Here are links to other pages about GitLab Dedicated around GitLab:

- Docs: Configure GitLab Dedicated
- Product: Glossary of Switchboard terms
- Infrastructure: GitLab Dedicated internal docs (GitLab internal only)
- CSM: Engaging with GitLab Dedicated Customers (GitLab internal only)

Handling tickets that are not about GitLab the product but related to how we handle the infrastructure

Handling GitLab Dedicated tickets should be approached the same way as other tickets: use the docs, the handbook and the various issue trackers to help address the customer request. Anything that is related to our provisioning of the tenant or how we manage the infrastructure requires a different approach:

- check the docs and the handbook
- point the customer to the GitLab Trust Center at trust.gitlab.com for inquiries related to compliance
- point the customer to their CSM for any questions not addressed by the GitLab Trust Center

Test and reproduction on GitLab Dedicated instance

GitLab Support has access to a GitLab Dedicated instance for testing and problem reproduction purposes. This instance can be accessed at the following URLs:

- GitLab: <https://dedicatedtestsandbox.gitlab-private.org>
- OpenSearch: <https://opensearch.dedicatedtestsandbox.gitlab-private.org/dashboards>
- Grafana: <https://grafana.dedicatedtestsandbox.gitlab-private.org>
- Switchboard: <https://console.gitlab-private.org/tenants/40>

To receive an invite, ask Armin, Brie, Daphne or Wei-Meng in supportgitlab-dedicated.

Conducting a test

When running a test on the GitLab Support Dedicated instance,

- consider whether your test can be conducted in an instance deployed via the Sandbox Cloud Realm
- communicate about it on the Slack channel supportgitlab-dedicated
- revert your changes when you are done

As the test instance is shared within the GitLab Support team, post a message at the beginning of a test with an estimate duration if the test is likely to impact the performance of the instance.

Upon test completion, revert your changes and use the emoji :done: to show the test is completed and the instance has been restored to the previous state.

This instance is deployed to the Test environment.

GitLab Duo is not supported on the test instance, see STM 6619 for more information.

Administrative access to a Dedicated instance

The GitLab Dedicated team does not have administrative access to the Admin Area in the GitLab application on Dedicated instances and neither does the GitLab Support team. Select individuals in the customer organization do have access to the Admin Area. Any support requests that require a GitLab instance administrator to make a change in the Admin Area, for example resetting 2FA, has to be performed by the appropriate team within the customer organization.

Sharing internal logs, data & graphs

We should not visually or physically share internal logs, data and graphs with GitLab Dedicated customers by default. Examples of things we should not share include, but are not limited to, screenshots of graphs, copied log entries, and raw log dumps.

For avoidance of doubt:

- Internal logs refers to logs not generated by the GitLab application.
- Internal data and graphs include all data available in Grafana.

Here is some background to understand why:

1. GitLab Dedicated comes with a Service Level Availability SLO, which if not met results in financial penalties for GitLab.
1. Capacity is limited for the GitLab Dedicated engineering teams as of October 2023. The teams want to spend the majority of their time on engineering tasks and avoid spending time answering non-critical customer questions.

Sharing internal logs, data and graphs without adequate context and explanation may cause customers to misinterpret the provided information, creating more work for all teams involved and, in the worst case, cause unnecessary damage to GitLab's relationship with the customer.

If you assess that sharing such internal logs, data and graphs with the customer would create results for the customer and for GitLab, consult with a Director of Support. Be aware that a formal process for this is still being defined, and that there will be delays as approvals are currently ad hoc.

Working with logs

Working with logs has been moved

Working with Grafana

Working with Grafana has been moved

View instance metadata

Use the Switchboard app. More information can be found in the Switchboard workflow.

Feature Flags are not supported

In GitLab Dedicated, feature flags are not supported, meaning we do not able enable/disable a feature flag for a Dedicated instance. When customers request feature flags to be modified in the GitLab Rails console, the GitLab Support team should:

- create or find an issue in the appropriate issue tracker about making this feature generally available (without a feature flag).
- notify the appropriate Product Manager in the issue with a comment that followed the feedback template.
- notify the customer that we don't keep tickets open even if the underlying issue isn't resolved.
- check whether the customer has any questions about the next steps.

Support team members with questions can check in the sptpoddicated Slack channel for additional guidance.

Feature proposals

GitLab Dedicated feature proposal issues should be created in the Public [gitlab-org/gitlab](https://gitlab-org.gitlab.io)

issue tracker. Mention the Product Manager when opening a feature proposal issue. Use the feedback template to register a customer's interest in the feature proposal.

Configuration changes

GitLab Dedicated uses the Cloud Native Hybrid reference architecture. Instance implementation and changes are done via the instrumentor project.

If it's an emergency, raise a Dedicated incident.

When any changes are required besides those listed below, raise an issue with `SupportRequestTemplate-GitLabDedicated`.

Inbound (Forward) PrivateLink Request

1. In the ticket, ask the customer to provide the required information. In this case, it's an IAM principal.

- The IAM principal must be an IAM role principal or IAM user principal.
- The IAM user principal has the following format:

`arn:aws:iam::<CustomerAWSAccountID>:user/user-name`. The IAM role principal has the following format: `arn:aws:iam::<CustomerAWSAccountID>:role/role-name`. Keep the format of these two in mind to avoid prolonging the ticket if an unexpected format is provided.

1. Open a new PrivateLink Request issue and confirm that the `support::request-for-help` label is added.

1. Add the IAM principal to the issue. The Environment Automation team will provide a Service Endpoint Name.

1. The customer will follow the steps in our documentation.

Outbound (Reverse) PrivateLink Request

Outbound PrivateLink allows any traffic from the GitLab Dedicated instance, or hosted runners for GitLab Dedicated to not be exposed to the public internet. While we advise for the onboarding to be completed, the customer can start these steps before.

1. Open a new PrivateLink Request issue and confirm that the `support::request-for-help` label is added.

- As a comment in the issue, request two Availability Zone IDs (AZ IDs) that can be used by the customer. For hosted runners, we only need a primary Availability Zone ID (AZ IDs).

1. Provide the IAM role Principal to the customer. It has the following format: `arn:aws:iam::<AWSAccountID>:role/reverseprivatelink@<tenantid>`. You will find this information in the Reverse Private Link IAM Principal row of the Tenant Details section in Switchboard. Alternatively, read the instructions in the issue created for information on how to find the `<AWSAccountID>` and `<tenantid>`.

1. Provide the two AZ IDs from the issue to the customer. An example AZ ID is: `use-az1` or `usw-az4`. Note: These are not AWS Zone IDs.

- Provide the two AZ IDs early in the ticket to avoid prolonging the ticket. The AZ IDs must be in the same region as the customer's tenant instance. The customer can then make the decision of which specific zones that can be used. AZ IDs are shared between different zones in a region but cannot be used outside of the region. For example, AZ IDs in us-west-1 cannot be used in us-west-2. Some of the zones in each reach share AZ IDs with other zones in the same region but you must work with the customer to find the overlap.

1. Ask the customer to provide the required information. In this case, it's a Service Endpoint Name, a list of AZ IDs they will be using (should match provided AZ IDs), and Domain Name (with one of two options).

- The Service Endpoint Name uses a reverse domain name notation and has the following format: `com.amazonaws.vpce.<region>.<vpce-svc-identifier>`

1. Fill in the issue with the information provided by the customer and follow next steps in the issue.

IP Allowlist Request

In most cases, customers should use Switchboard to update the IP allowlist for their GitLab Dedicated instance. If this is not possible:

1. Ask the customer to provided the required information in the ticket. In this case, it's a comma-separated list of IP addresses.

1. Open a Request for Help issue and confirm that the `support::request-for-help` in the GitLab Dedicated issue tracker.

SCIM / OIDC with IP Allowlist request

Customers who use the IP allowlist may request to enable the SCIM or OIDC endpoints to the internet. This is a simple on/off toggle but must be performed by the Environment Automation team:

1. Open a Request for Help issue in the GitLab Dedicated issue tracker. (Confirm that the `support::request-for-help` label has been applied)

SAML Request

1. Ask the customer to provided the required information in the ticket. In this case, it's a SAML configuration block or can be a list of information provided by a customer.

1. Open a new SAML Config Request issue and confirm that the `support::request-for-help` label is added.

1. Add the customer provided information and match it with the required formatting.

Application Logs Request

1. In the ticket, ask the customer to provide the required information. In this case, it's an IAM principal.

- The IAM principal must be an IAM role principal or IAM user principal.

1. Open a Request for Help issue in the GitLab Dedicated issue tracker.
1. Provide the IAM principal to the Environment Automation team.
1. Provide the name of the S3 bucket to the customer.

Filing issues

In cases where Customer Support need to interact with Dedicated engineers to gather information or debug a problem at tenant's request (when Grafana or OpenSearch do not suffice), raise an issue in the Request for Help issue tracker using the Request for Help template.

RFH have an SLA of three working days for all severity levels. For severity 1 and 2 issues based on Support definition, consider raising a Dedicated incident. Ask in Slack supportgitlab-dedicated if you are unsure.

During the course of the investigation, you may realize that you need to escalate a Request for Help (RFH) issue to another team. You should follow the existing process to formally request help from another group in the GitLab Development Team. When doing this:

- Summarize the investigation thus far and make sure the ask for the team you are escalating to is clear
- Apply the ~workflow-infra::Escalated label to the RFH issue in the GitLab Dedicated issue tracker

Handling GitLab Dedicated emergencies

GitLab Dedicated customer emergencies follow the same definitions and exceptions as regular customer emergencies.

If the customer is reporting an availability or performance issue:

1. Check the GitLab Dedicated incident management tracker for relevant open incidents.
1. If there is an open incident:
 - Inform the customer that the GitLab Dedicated infrastructure team is actively investigating.
 - Get in touch with the Dedicated SRE on-call and determine if the customer needs to be involved with troubleshooting.
 - Assist the customer and the Dedicated SRE as necessary.
1. If there isn't an open incident, raise a Dedicated incident.

Raise a Dedicated incident

When raising a GitLab Dedicated incident, you must complete all three steps:

1. Follow the Dedicated On-Call runbook.
1. In the incident channel that is automatically created on Slack, provide a summary of the current state.
1. Escalate to the Dedicated Engineer On-Call by following the escalation process

You are now done raising the incident!

Engaging the GitLab Dedicated CMOC

If the nature of the emergency reaches the point where we only need to provide async status updates to the customer, consider engaging the GitLab Dedicated Communications Manager on Call to take over.

Troubleshooting tips

Tagging logs while running tests

Customers can add a custom identifier, such as the ticket ID, to the user-agent field when testing. This makes it easier to filter logs related to the test.

For example:

```
bash
curl -k -vvv -A"GitLabSupport012345" "https://tenant.gitlab-dedicated.com/users/signin"
```

SECTION: support/workflows/dedicated_cmoc.md

Introduction

The GitLab Dedicated Communications Manager on Call (GDCMOC) is an async role with the purpose of contact outreach from GitLab to a GitLab Dedicated customer. There is no expectation on the GDCMOC to be performing troubleshooting responsibilities. It involves communicating with team members in slack or GitLab issues, and then relaying the information to the customer in a Zendesk ticket to keep them up to date. GDCMOCs do not need to dedicate all their focus to monitoring the relevant threads/issues, but they should regularly check communication threads for updates that need to be communicated to customers.

The GDCMOC rotation currently uses the GitLab.com CMOC rotation to determine who is oncall. This means that when you go oncall as a GitLab.com CMOC, you will also be the GDCMOC. This may change at a later date if it's determined we need to split this out to its own rotation.

Engaging the GDCMOC

The GDCMOC can be paged via Slack or directly via PagerDuty.

- Slack: Using the /pd trigger command in Slack, select Incident Management - GDCMOC in the Impacted Service modal. Fill in the Title and click Create.
- PagerDuty: From the Incident Management - GDCMOC page, click New Incident. Fill in the Title and click Create.

While the Description field is optional, you should always include a link to an issue or ticket with more information for the engineer who is being paged.

There is additional information about engaging the GDCMOC in the on-call runbook for the GitLab Dedicated team.

Acknowledging a GitLab Dedicated Contact Request

If an urgent contact request is needed, you will be paged via PagerDuty. Start by marking the PagerDuty alert as acknowledged. This can be done through the mobile app, web interface or PagerDuty App in the supportgitlab-dedicated Slack channel.

The description in the PagerDuty alert should contain details about an issue, or a slack thread you need to follow. Follow any communication threads, and let the Dedicated Incident team know you are available to assist. You should be provided with the details of the request. If you're unsure, check the GitLab Dedicated incidents issue tracker or ask in the gdedicated-team slack channel.

Creating a new outreach ticket

Follow these steps to create a new Outbound Contact Request ticket for the customer.

1. Find the Switchboard (production) tile in Okta and login to Switchboard.
1. You should see the Tenants page when logged in. Find the relevant tenant and click Manage.
1. Expand the Cloud Account Config section, and look for the Primary Region. This should tell us which region the customer is based in. See the AWS docs if you're unsure of the AWS region code. Make a note of the region.
1. Search for the Customer Communications section, and expand it. You should see values for Operational email addresses and Customer CSM. You will need these values when creating the ticket.
1. Follow the instructions here to create a Zendesk ticket for the outbound request.
 1. For the subject of the ticket, use the following template: GitLab Dedicated Notice: <description>.
 1. Apply the macro General::Outbound Contact Request
 1. For the ticket requestor, use the first operational email address listed.
 1. CC the other operational email addresses and the Customer CSM.
 1. Set the Preferred Region for Support to the region similar to where the tenants' Primary Region is located.
 1. Add a dedicatedcontactedrequest tag to the ticket.
 1. Set the "Support Resolution Codes" to Incident.
1. Assign the ticket to yourself.
1. After sending the initial outreach message to the customer, mark the PagerDuty alert as resolved. The alert's purpose is specifically to engage the GDCMOC to start communication.

Keep the customer informed

- Follow any issues or slack threads for any updates to the incident. If there's anything worth sharing with the customer (such as "we found a possible lead" or "We are currently digging into the sidekiq logs"), do so in the ticket. If you're unsure of anything, ask the GitLab Dedicated Engineer OnCall or Incident Manager for clarification.
- Work with the customer to set expectations about the frequency of updates, especially if you are the GDCMOC within the same region as the customer. They will likely expect more updates during their regional business hours.

- If we proceed with lower frequency updates, the important thing is that we communicate our expected update frequency to them. For example, we can let the customer know that during their regional business hours, we will provide an update every 1-2 hours, and during their non-regional hours we will update them if there is anything substantial to share.
- Keep in mind the information that we should not share with the customer
- If you'd like a second pair of eyes to review messages before sending them out to customers, refer to the table below to find an appropriate DRI.
- Approval of message content is required for security-related communications.
- Approval is optional for all other communication.

Communication type	Who reviews content?	Who approves content?
Non-security out-of-band maintenance	SRE	Optional
Security-related out-of-band maintenance	SIRT	SIRT
Incident communication	SRE / Incident manager	Optional
Other urgent communication	It depends	Optional

Closing the Outreach Ticket

The outreach ticket's scope is specifically for customer communication about a particular incident or maintenance. Once the communication about the incident/maintenance is complete (For example, maintenance is finished or the incident is resolved), you should:

1. Send a final update to the customer confirming the completion.
1. Close the outreach ticket.
1. Add a brief internal note summarizing the communication timeline (optional).

Note: If the customer responds with follow-up questions after closure, create a new ticket to handle those inquiries separately from the original outreach communication.

Guidelines

- You do not need to focus all of your attention on looking for updates. As a guideline, check the issues, slack threads, and customer ticket for any updates every 30 minutes if you are the GDCMOC in the same region as the customer.
- Consider sending the customer a summary so far if it's been multiple hours since the last substantial update. For example: "To summarise so far, we initially looked into A and B, now we're investigating C", or "We found something in the logs to indicate the problem may be with X. I will share further details if the team finds anything substantial."

Concurrent incidents

Support Engineers are not expected to manage multiple incidents. If a concurrent GitLab.com incident or GitLab Dedicated contact request comes in, engage with the Support Manager oncall to help find cover for the new incident.

You can ping the Support Manager oncall in slack with @support-manager-oncall.

Handover

Follow the End of Shift Handover Procedure from the CMOC workflows. Make the ingress GDCMOC aware of any issues, slack threads or tickets they should CC themselves on. Assign the Zendesk ticket used for communication to the next CMOC.

Emergency maintenance

A security vulnerability fix might result in emergency maintenance for GitLab Dedicated environments.

NOTE:

"Emergency maintenance" refers exclusively to security-related maintenance. Maintenance that happens outside of the weekly scheduled maintenance window should be referred to as "out-of-band maintenance"

In such cases, the GitLab Dedicated CMOC will be called upon to send initial notification to customers using the Support::Dedicated::Emergency Maintenance - Security macro. The following information should be provided:

- The list of customers to contact.
- The date and approximate time (in UTC) that emergency maintenance was carried out.
- Whether the customer's GitLab Dedicated environment was impacted and details if any.
- A brief summary that can be used in the subject line of the ticket/message.

SECTION: support/workflows/dedicated_instance_health.md

Grafana data is primarily for internal use

Monitoring and observability graphs from Grafana should not be shared with customers by default.

If you think sharing graphs would benefit the customer, please read [Sharing internal logs, data & graphs](#).

Accessing Grafana

The credentials for accessing the Grafana instance associated with each GitLab Dedicated tenant are stored in the GitLab Dedicated - Support 1Password Vault. As with the OpenSearch instances, the passwords are referred to by customer number. Use the information in the internal note in the ticket and the subdomain in the website field in 1Password to find the credentials for the correct GitLab Dedicated tenant.

Preprod deployments

Use the GitLab Dedicated Preprod switchboard to find links to the Grafana dashboard for a specific customer.

Finding Dashboards

When logging in for the first time, dashboards are not immediately visible and you will be greeted by a Grafana welcome screen. To find the dashboards:

1. In the left pane, click on Dashboards.
1. The Dashboards page has a searchable list of all the dashboards available to you
1. The Triage dashboard is the best starting point, especially if you are new to Grafana.

Grafana tips

The General / Triage dashboard is most useful for an emergency as it has the pods all laid out in a single view. By default it has 6 hours of data. It is helpful for finding blips & dips. Use this data to correlate to other dashboards.

If there is a point of interest on a given graph in the dashboard, you can click + drag to zoom in on the graph.

Remember that Grafana is used for visualizing issues and spotting problems. It won't tell us directly what is wrong. You must correlate to the logs to find the exact problem.

As an example, the Triage dashboard may show that webservice errors are increasing. Use this to correlate an approximate time and filter out the other logs by the `kubernetes.labels.app` to find the error for just webservice.

The General / Triage dashboard contains information about Service Apdex (application performance index), Service Error Ratio, RPS (requests per second) and Saturation information for each service included in that dashboard. In general, you should notice that increases in the Service Error Ratio correspond to decreases in the Service Apdex. Read more about GitLab Dedicated SLAs.

SECTION: support/workflows/dedicated_logs.md

Working with logs

Support can access GitLab Dedicated tenant logs through our OpenSearch infrastructure. See [Accessing logs to get started](#). OpenSearch can be used like Kibana but read about [searching logs](#) for information on the differences.

When working on a GitLab Dedicated ticket, prioritize asking for information that will help identify applicable log entries. It is best to start collecting this information as early in the ticket as possible. The specific kinds of information will vary depending on the problem you are trying to solve but username, project path, project ID, exact date and time with time zone, correlation ID and outgoing IP address are all good examples.

The logs in OpenSearch will all be presented in the UTC time zone, regardless of the customer's time zone.

Preprod deployments

Use the GitLab Dedicated Preprod switchboard to find links to OpenSearch logs for a specific customer's Preprod environment, when applicable.

Log requests older than 7 days

If the customer requests logs for a period older than 7 days, a security issue should be created. Follow the same procedure as the Security - log request workflow.

Identifying tenants

Each customer has a dedicated set of credentials needed for examining logs in OpenSearch. The credentials and the URL for that customer's OpenSearch instance are stored in the GitLab Dedicated - Support 1Password vault. Each customer is noted by a three word Internal reference in the vault, so you must refer to the <tenant name> to identify the proper credentials to use for a customer. This is used as part of the accessible URL, such as: opensearch.<tenant name>.gitlab-dedicated.com. You should use Switchboard as the single source of truth for identifying the Internal reference for a tenant based on the GitLab Dedicated instance URL.

Accessing logs

To access the logs for a specific tenant find the credentials stored in the GitLab Dedicated - Support Vault, and access the corresponding tenant URL listed there. Once in the tenant's OpenSearch site:

1. Select "Global" tenant
1. Choose "Discover" at the sidebar under OpenSearch Dashboards
1. On the next screen, you should see logs. Make sure that index gitlab- is selected.

It is recommended to start with the gitlab- index because it has a timestamp field. It shows a useful skyline graph and allows fo